



Mekelle University

Mekelle Institute of technology

Mini Project : Digital Rental Platform for Property Owners and Renters in Mekelle

Prepared by:

Name

ID

1. Yordanos Hagos.....mit/ur/148/12
2. Axumawit Leake..... mit/ur/018/12
3. Halefom Haregot..... mit/ur/070/12
4. Shewit Grmay mit/ur/268/12
5. Niema Kedir.....mit/ur/168/12

Supervisor: Tr. Gebregiorgis B

Submission Date : June 4, 2025

Abstract

The ongoing urbanization in Mekelle, Ethiopia, has led to an increased demand for rental housing. However, the traditional rental process in the city remains largely inefficient, relying heavily on brokers, word-of-mouth referrals. These practices often lead to misinformation, increased costs, lack of trust, and a time-consuming search process for both renters and property owners. Recognizing these limitations, our project proposes the design and development of a centralized web-based rental platform tailored to the local needs of Mekelle's residents.

This platform aims to provide a reliable, user-friendly, and transparent interface where property owners can list available rental units, and renters can search and filter listings based on criteria such as price, location, and number of rooms. By eliminating the role of intermediaries, the platform reduces financial burdens on users and enhances the accuracy of rental information. The system architecture is built using Vue.js for the frontend, Node.js for the backend, and MongoDB as the database, ensuring scalability, responsiveness, and effective data handling.

Through structured phases of requirement analysis, UI/UX design, full-stack development, and deployment, the platform was brought to a functional stage and deployed using Netlify. The project also incorporated a use case analysis, testing strategies, and team collaboration to achieve the outlined goals. While the initial version does not include features like messaging or payment integration, it lays the foundation for future enhancements such as AI-powered recommendations, secure payment systems, and identity verification tools. Ultimately, this platform aspires to simplify the rental process in Mekelle and serve as a prototype for broader implementation across Ethiopian cities.

Acknowledgment

We are also sincerely grateful to Haftom Haregot from VITE Technologies PLC, a former MIT student, for his support and guidance during the development of this project. His mentorship contributed significantly to our understanding of practical implementation and real-world problem-solving.

Table of content

<u>1.</u>	Introduction.....	6
<u>2.</u>	Problem Statement	7
<u>3.</u>	Objective	7
<u>4.</u>	Literature review	8
<u>5.</u>	Methodology	8
5.1	Development phase.....	8
5.1.1	Requirement gathering	9
5.1.2	UI/UX design	9
5.1.3	Development	10
5.1.4	Testing	10
5.1.5	Deployment	10
5.2	Implementation	11
5.3	Usecase diagram	12
<u>6.</u>	System design.....	13
<u>7.</u>	Result and discussion	13
7.1	Property listing form	14
7.2	Search and filter interface	15
7.3	Property detail view.....	15
<u>8.</u>	Team roles and contribution	16
<u>9.</u>	Challenges and lesson learned	17
<u>10.</u>	Conclusion	18
<u>11.</u>	Future work	19
<u>12.</u>	References	20

Abbreviations and Acronyms

- **UI** – User Interface
- **UX** – User Experience
- **MVP** – Minimum Viable Product
- **JWT** – JSON Web Token
- **CRUD** – Create, Read, Update, Delete
- **API** – Application Programming Interface
- **ORM** – Object Relational Mapping
- **DB** – Database
- **SPA** – Single Page Application

1. Introduction

The rental ecosystem in Mekelle, a rapidly growing city in northern Ethiopia, faces significant inefficiencies and obstacles that impact both renters and property owners. Traditionally, the search for rental properties relies heavily on unstructured methods such as word-of-mouth referrals, physical visits to neighborhoods, local brokers, and informal listings shared through social media platforms. These outdated approaches result in multiple challenges, including high time consumption, financial strain, and a lack of trust and transparency between the involved parties.

Many renters, particularly students and professionals moving into the city, are often met with difficulty in finding reliable and accurate information about available properties. At the same time, property owners are limited in their ability to effectively market and manage their listings without incurring additional costs through broker commissions.

Our project addresses these problems by introducing a digital rental platform designed specifically for the Mekelle housing market. The platform streamlines the rental process by enabling property owners to list their properties online and allowing renters to search and filter listings based on their preferences. It reduces the need for brokers and physical legwork, offering a more efficient, transparent, and cost-effective way to connect renters with property owners.

The introduction of such a digital system is both timely and practical given the increasing access to the internet and smartphones among the local population. The solution is built using modern web development technologies that support scalability and ease of use. In doing so, the platform not only modernizes the rental process in Mekelle but also creates a model that could be expanded to other cities facing similar challenges. Moreover, it sets a foundation for the potential integration of advanced features in the future, such as secure payment systems, user verification tools, and AI-driven property recommendations.

By bridging the gap between traditional practices and digital innovation, this project contributes to the broader goal of enhancing the real estate sector in Ethiopia through technology-driven solutions. It empowers both renters and property owners with better tools, improves transparency in the housing market, and encourages trust and fairness in rental transactions.

2. Problem Statement

The current rental process in Mekelle presents multiple challenges for individuals searching for houses to rent. Most people rely on local brokers, word-of-mouth to find available properties. This method is not only time-consuming but also lacks transparency and reliability.

Finding a suitable rental property requires significant effort. Renters often have to physically visit multiple neighborhoods or contact numerous brokers, which consumes both time and energy. Comparing rental options is difficult as there is no centralized platform where users can browse and evaluate multiple listings based on criteria like price, location, and amenities.

Additionally, there is often a mismatch between what brokers describe and the actual condition of the property. This miscommunication leads to disappointment and mistrust, especially when renters realize the property does not meet their expectations. The cost of using brokers is also high, with some charging up to a month's rent as commission.

These issues highlight the need for a centralized digital solution that simplifies the rental process, reduces costs, and improves trust between renters and property owners.

3. Objectives

General Objective: To build a responsive and user-friendly web-based platform that modernizes the rental housing process in Mekelle.

Specific Objectives:

- To develop a digital system that allows renters to search and filter verified property listings efficiently.
- To enable property owners to register, upload, and manage their rental listings online.
- To reduce or eliminate the dependency on traditional brokers.
- To enhance transparency, accessibility, and trust within the rental ecosystem.

4. Literature Review

The advancement of digital technologies has transformed how rental markets function across the world. In developed countries, platforms such as Airbnb [1], Zillow [2], and Trulia have become key players in property rental and management. These platforms centralize listings, provide high-quality images, offer filtering tools, and include verified user reviews, which altogether enhance trust, convenience, and decision-making for renters. Users can easily compare properties, schedule viewings, communicate with landlords, and even make payments—all from a single interface.

In the context of developing nations, similar progress has been made, though often adapted to the specific local challenges. In African countries like Nigeria and Kenya, platforms such as PropertyPro [3] and BuyRentKenya [4] have started reshaping how people find and list properties. These platforms demonstrate how digital tools can reduce reliance on brokers, eliminate inflated commissions, and improve transparency.

However, implementing such platforms is not without its challenges. Barriers such as internet accessibility, smartphone availability, digital literacy, and trust in online transactions still hinder widespread adoption. Successful platforms often address these issues by integrating verification systems, customer support, and easy-to-navigate interfaces tailored for users with minimal digital experience.

For Mekelle and similar cities in Ethiopia, where digital infrastructure is growing but still maturing, the adoption of a tailored digital rental platform can be transformative. Drawing inspiration from both global and African models, our project takes these best practices and adapts them to suit local realities. By focusing on verified listings, intuitive filtering, and a broker-free process, the platform aims to improve rental efficiency and user trust while laying the groundwork for future features such as payment gateways, AI recommendations, and identity verification systems.

5. Methodology

The methodology adopted for this project followed an incremental and iterative approach, allowing for continuous improvements and testing at every phase of development. It was structured to cover the complete software development life cycle—from requirement analysis to design, implementation, testing, and deployment—ensuring a high-quality, user-focused final product.

5.1 Development Phases

5.1.1 Requirement Gathering

This phase involved understanding the key issues and expectations of the two primary user groups: property owners and renters. The team conducted informal interviews to identify the pain points of the current rental system in Mekelle. The data collected highlighted a lack of centralized listings, trust issues, and inefficient communication, which helped shape the functional and non-functional requirements of the system. Based on the feedback, the essential features were defined: user registration, property listing, search and filter options, and secure access control.

5.1.2 UI/UX Design

Design was a critical aspect of the project and aimed to ensure that the platform would be intuitive, visually appealing, and accessible to users with varying levels of digital literacy. The design phase began with low-fidelity wireframes to map the structure and layout of the major screens, such as the homepage, search interface, and property detail views. These wireframes were iteratively refined into high-fidelity mockups using Figma.

Special emphasis was placed on accessibility and usability. The platform's color palette, typography, and spacing were selected to enhance readability, while interactive elements like buttons, inputs, and cards were made prominent and easy to use on desktop. User-centered design principles were followed to minimize friction during key tasks such as searching for properties, submitting listings, or logging in.

The team also created interactive prototypes to simulate user flows and test navigation efficiency. These prototypes were shared with potential users for informal feedback, which helped in refining visual hierarchy and improving layout clarity. The goal was to reduce user cognitive load and make the platform as intuitive and self-explanatory as possible.

Key Screens Designed:

- Home page with featured listings and navigation
- Filter/search results page with dynamic filters
- Login and registration forms with validation hints
- Property submission form with multi-step input flow
- Property detail view with images, amenities, and owner details

5.1.3 Development

Development was a collaborative process involving close coordination between frontend, backend, and database teams. Each team member took ownership of specific modules, ensuring consistent progress across all components. Version control was handled using Git and GitHub, with daily commits and weekly integration merges to reduce conflicts and maintain a clean codebase.

The frontend development prioritized performance and responsiveness. Vue.js was used to implement dynamic data binding, component reusability, and routing between views. Tailwind CSS allowed for rapid styling, resulting in a clean and modern interface. Axios was used to manage API calls, enabling seamless data flow between frontend and backend. Dynamic search and filter features were implemented with reactive state management to give users real-time results without page reloads.

On the backend, Node.js and Express.js were employed to build secure RESTful APIs for user registration, authentication, listing management, and search functionality. The team enforced security best practices such as using bcrypt for hashing passwords, validating all user inputs, and setting up CORS for controlled cross-origin access.

MongoDB was used to store user accounts, property listings, and search indexes. Schema design included embedded documents for property images and indexed fields to optimize search performance. The backend was designed to be modular, making it easier to maintain and extend in future iterations.

Extensive console logging and error handling were embedded throughout the code to simplify debugging. The system was tested for performance under various query conditions and ensured to support concurrent users accessing and searching property data simultaneously.

5.1.4 Testing

Testing was performed in two major phases:

- **Unit Testing:** Each component and feature was tested in isolation to ensure functionality.
- **Integration Testing:** Ensured that the frontend and backend components interacted correctly.
- **Manual Testing:** Individuals were asked to test the application and give feedback on usability, bugs, and suggested improvements.

Test cases focused on:

- User registration and login flows
- Property submission and search accuracy
- Form validations and error handling
- Interface responsiveness and compatibility

5.1.5 Deployment

The final application was deployed using Netlify, allowing public access via a shareable link. Continuous deployment was enabled to reflect changes as the team updated features. Before going live, a final round of testing was conducted in the production environment to validate stability and performance.

Deployment Highlights:

- Hosted frontend on Netlify for quick load time and ease of access
- Configured environment variables and database settings securely
- Ensured availability across multiple devices and browsers

5.2 Implementation

Implementation was carried out in stages, aligned with the modular structure of the system. This phase translated design into code, transforming ideas and wireframes into a functional and interactive platform. The entire system was broken into two main layers: frontend and backend, each of which was developed, tested, and refined separately before being integrated.

The frontend layer was implemented using Vue.js, a JavaScript framework known for its reactivity and performance in building user-centric web applications. Vue Router was used for navigating between components, while Vuex was considered for state management to streamline user session handling and search filters. Components were modularized for scalability, and each feature—such as search filtering, property cards, login, and listing submission—was handled in reusable components.

The backend was developed using Node.js with the Express.js framework to manage server-side logic, define routes, and handle HTTP requests. RESTful APIs were built to serve property data to the frontend, perform user authentication and support filtering logic based on user queries. Security practices such as input sanitization, password hashing using bcrypt, and role-based access control were applied to ensure safe operation.

MongoDB served as the primary database, chosen for its flexibility in managing varied and evolving property data. The Mongoose library was used to define schemas for users and property documents, enforce data integrity, and perform efficient database queries. The team also designed the database to support future scalability for potential features like payments and user reviews.

Features Implemented:

- User registration and login system with validation
- Secure password hashing and token-based user authentication (JWT)
- Property listing form with dynamic image upload handling
- Property display cards showing images, location, price, and description
- Advanced filtering by location, price range, and number of rooms
- Responsive design optimized for desktop views
- Property updates reflecting changes in the database

Technology Stack:

- Frontend: Vue.js, Tailwind CSS
- Backend: Node.js, Express.js
- Database: MongoDB, Mongoose
- Deployment: Netlify
- Tool for UI design: Figma

5.3 Use Case Diagram

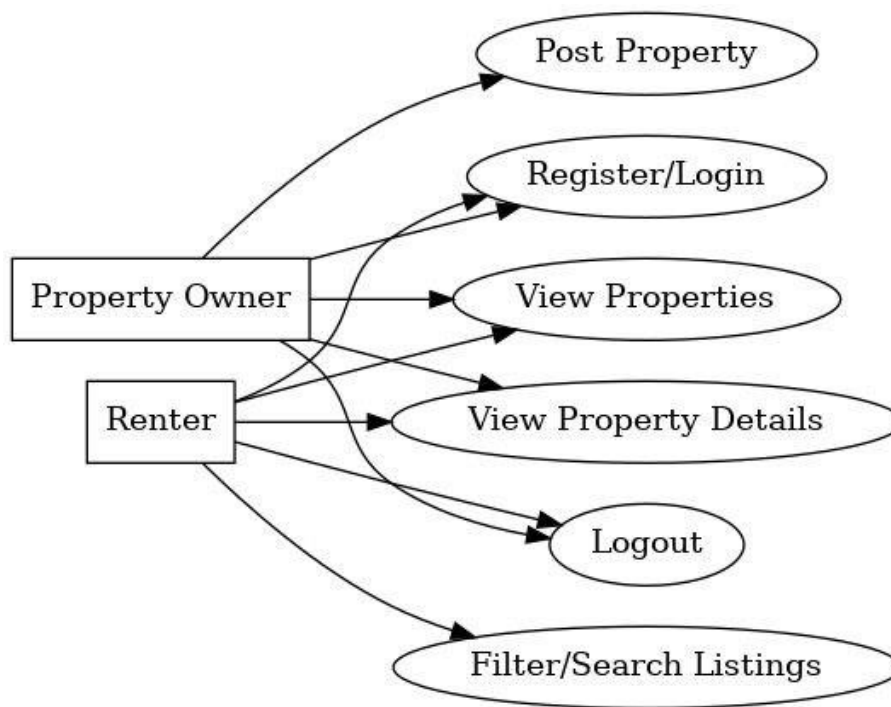
The use case diagram below outlines the major interactions between users (actors) and the system. It provides a high-level overview of the platform's functional requirements.

Actors:

- Renter
- Property Owner

Use Cases:

- Register/Login
- View Properties
- Filter/Search Listings
- Post Property
- View Property Details
- Logout



6. System Design

The architecture of the platform was carefully designed to balance performance, scalability, and usability. The system is based on a client-server model and uses a RESTful API approach to manage communication between the frontend and backend.

Frontend Design: The frontend of the platform was built using Vue.js, a progressive JavaScript framework known for its responsiveness and ease of integration. Vue components were modularized for maintainability, and Tailwind CSS was used to maintain design consistency. The interface was designed to work efficiently across desktop platform, focusing on minimal clicks and fast load times.

Backend Design: The backend, developed using Node.js and Express.js, handles all server-side logic, routing, and user authentication. RESTful APIs serve as the communication bridge between the frontend and backend. The backend is also responsible for validating data, securing user credentials using JWT and bcrypt, and handling database operations.

Database Design: MongoDB was used as the database for its flexibility and performance. Collections were created for users, property listings, and images. Each listing document includes nested data structures for amenities, images, and owner references. Indexes were used to optimize search queries and filtering performance.

Security Measures:

- JWT-based user authentication
- Password hashing using bcrypt

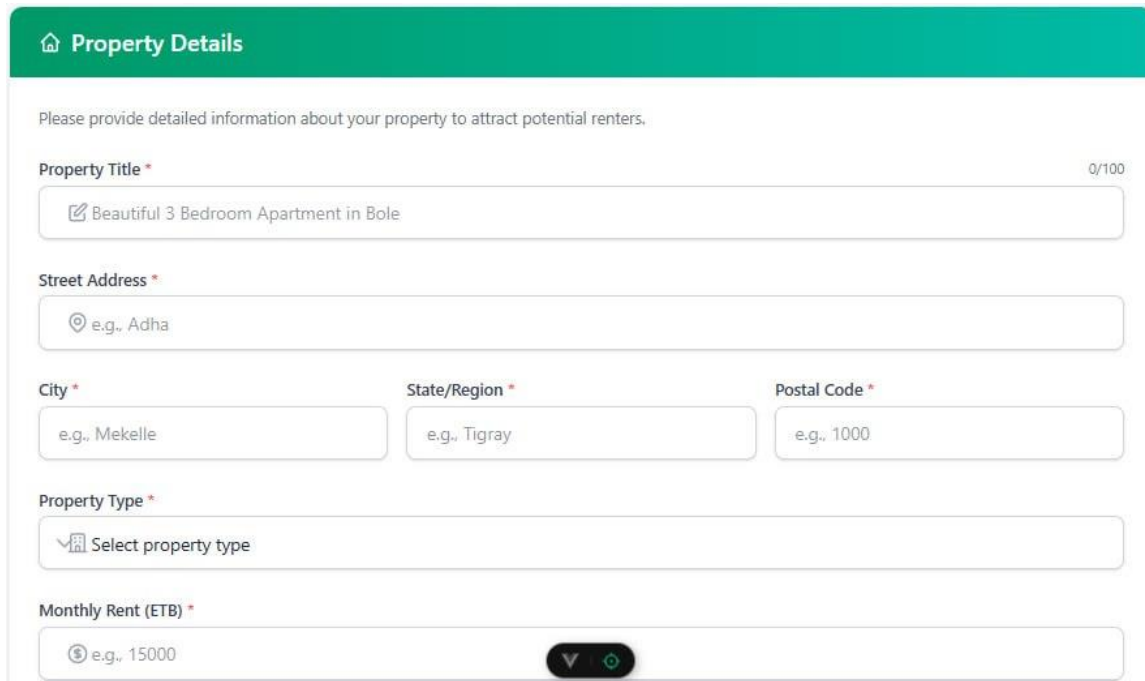
Scalability: The architecture supports horizontal scaling, with the potential to add additional services such as payment processing, chat systems, and analytics modules.

7. Results and Discussion

To better illustrate the outcomes of our project, this section includes not only a narrative summary of system performance and feedback but also visual evidence from the deployed platform. Images have been embedded to showcase the real implementation and interface components built during development. These images allow evaluators and readers to visualize the actual application in action, including how users interact with it and what functionalities are available.

The figures below show key screens such as the homepage, listing form, filtering interface, and detailed property view. Each image is accompanied by a description that explains its relevance to user interaction, interface design, and system functionality.

7.1 Property Listing Form



The screenshot shows a web form titled "Property Details" with a teal header. Below the header, a message reads: "Please provide detailed information about your property to attract potential renters." The form contains several input fields: "Property Title *" with a character count of 0/100 and a placeholder "Beautiful 3 Bedroom Apartment in Bole"; "Street Address *" with a location pin icon and placeholder "e.g., Adha"; "City *" with placeholder "e.g., Mekelle"; "State/Region *" with placeholder "e.g., Tigray"; "Postal Code *" with placeholder "e.g., 1000"; "Property Type *" with a dropdown menu showing "Select property type"; and "Monthly Rent (ETB) *" with a currency icon, placeholder "e.g., 15000", and a range slider.

Figure 6.2: Listing form

This screen allows property owners to submit detailed rental listings. It contains form fields for the property title, description, number of rooms, price, and location. Owners can upload multiple images to showcase the property visually. Input validation ensures that all required fields are completed correctly. The form was designed to be user-friendly and guide owners step-by-step through the submission process

7.2 Search and Filter Interface

The interface is titled "Find Your Perfect Property". It features a search bar with the placeholder text "Search by title, location, or keywords..." and a green "Search" button. Below the search bar are four filter categories: "Property Type" (set to "All Property Types"), "Bedrooms" (set to "Any Bedrooms"), "Location" (set to "Any location"), and "Max Price" (set to "No max"). Each filter has a dropdown arrow.

Figure 6.3: Filtering method

This interface enables renters to filter available listings based on criteria such as location, price range, and number of rooms. It dynamically updates the listings in real-time as filters are adjusted. This feature improves the user experience by minimizing the time required to find suitable properties and making the search process more efficient. The clean layout ensures renters can easily scan and compare listings.

7.3 Property Detail View

The property detail view for a "4 bedroom house" in "adha, mekelle, tigray". It includes a "Back to Listings" link. The main image shows a large house with a white wall and a black metal gate. Below the main image are three smaller thumbnail images. To the right, the "Property Details" section shows a price of "\$30,000/month", availability from "June 3, 2025", and a "For Rent" button. Below this, three boxes display "4 BEDROOMS", "4 BATHROOMS", and "200 SQ FT". Further down, the "Property Type" is "Apartment", the "Location" is "adha, mekelle, tigray", and the "Available From" date is "June 3, 2025". A "View on Map" button is at the bottom.

Figure 6.4: Property detail

This image presents the full details of an individual property. It includes high-quality images, pricing information, exact location and owner's information. A

contact section is available for users to reach out to the property owner. The layout provides clarity and is optimized to support desktop screen sizes, offering an ideal browsing experience.

The development and deployment of the digital rental platform yielded promising outcomes aligned with the goals set during the project planning phase. The system was successfully implemented using a full-stack approach with Vue.js for the frontend, Node.js for the backend, and MongoDB for the database. It was deployed on Netlify and made publicly accessible.

User Experience and Functionality The user interface proved intuitive during informal testing, with users appreciating the clean layout and fast response times. Renters could filter listings based on location and price, while property owners could upload listings with relevant details and images. The responsiveness of the application ensured usability across various device types including smartphones, which is important for accessibility in regions with limited desktop access.

Performance and Stability The system handled CRUD operations efficiently, and the backend API structure enabled seamless communication between components. Tests on concurrent access showed stable performance, even when multiple users interacted with the system simultaneously. Deployment on Netlify allowed for fast loading times and uninterrupted access during demonstrations.

Feedback and Observations Manual testing with peers and sample users highlighted the platform's usability and usefulness. Suggestions for future improvement included the addition of messaging features between renters and property owners, payment gateway integration, and enhanced location-based filtering.

the platform met all minimum viable product goals and demonstrated its potential to improve the rental experience in Mekelle.

8. Team Roles and Contributions

The project was completed through effective collaboration and a clear division of responsibilities among team members. The frontend team focused on designing an intuitive user interface and implementing it using Vue.js to ensure a seamless user experience. The backend development involved creating secure APIs and handling user authentication using Node.js, while the database team managed data modeling and integration using MongoDB. Additionally, tasks such as requirement analysis, documentation, and testing coordination were handled to ensure the platform aligned with the project goals and functioned reliably. Overall, the team's combined efforts led to the successful development and deployment of the home rental platform.

- **Yordanos Hagos** : UI/UX design and Frontend development using Vue.js.
- **Axumawit Leake** : UI/UX design and Frontend development using Vue.js.
- **Halefom Haregot** : Backend API and authentication with Node.js..
- **Shewit Grmay** : MongoDB data modeling and backend integration.
- **Niema Kedir** : Requirement analysis, documentation, and testing coordination.

9. Challenges and Lessons Learned

Throughout the course of the project, the team encountered several challenges that tested our ability to adapt, communicate, and manage time effectively. However, each obstacle also provided valuable learning opportunities that enriched our understanding of real-world software development.

Challenges:

- **Collaborative Coding Issues:** Managing code contributions from multiple developers using Git initially led to frequent merge conflicts, which slowed progress and required learning better collaboration techniques.
- **Backend Logic Bugs:** During development, the backend occasionally produced unexpected behavior due to improperly validated inputs or asynchronous operations, which required extra debugging.
- **User Interface Consistency:** Maintaining a consistent and professional UI across different modules was difficult without centralized style definitions early in the project.
- **Security Practices:** Implementing secure password handling and route protection using JWT tokens presented challenges, especially when managing authenticated routes and session states.
- **Search and Filter Logic Complexity:** Creating effective filtering features that returned accurate search results demanded precise backend querying and frontend state synchronization.

Lessons Learned:

- **Start Small, Iterate Fast:** Creating a minimal viable product first allowed us to test core functionalities early and improve them based on feedback.
- **Design-Driven Development:** Starting with UI/UX mockups and user flows helped align team efforts and ensured clarity in what needed to be built.

- **Value of Version Control:** Using Git and GitHub for collaborative development taught us the importance of version control, branching strategies, and resolving conflicts responsibly.
- **Clear Documentation:** Writing clean and well-documented code made it easier for all members to understand and build upon each other's work.
- **Feedback Matters:** Informal testing and feedback sessions with users provided perspectives that guided real improvements in usability and feature prioritization.
- **Collaboration Tools Are Essential:** Platforms like GitHub and Figma made it possible to collaborate remotely and manage different aspects of the project efficiently.

In summary, this project was not only a technical exercise but also a significant team-learning experience. The challenges we encountered helped us grow as developers, designers, and collaborators, and the lessons we gained will serve as a foundation for future software development endeavors.

10. Conclusion

The successful design and implementation of a web-based rental platform for Mekelle demonstrates the potential of digital technology to address long-standing challenges in the housing sector. By centralizing rental listings, enabling property owner participation, and empowering renters with powerful search and filter tools, the platform has proven effective in improving transparency, reducing dependency on brokers, and streamlining the rental process.

This project not only achieved its minimum viable product goals but also provided valuable experience in full-stack development, UI/UX design, team collaboration, and agile methodology. Informal testing and feedback confirmed the platform's usability and relevance to the local context. The work lays a solid foundation for future enhancements, such as secure online payments, direct messaging between renters and owners, and location-based recommendations powered by AI.

Ultimately, this project has shown that scalable, inclusive, and user-friendly solutions can emerge from student-led initiatives to solve real problems. It reflects the team's commitment to innovation, problem-solving, and technological advancement, serving as a model for future digital initiatives targeting the Ethiopian rental market and beyond.

11. Future Work

While the current version of the home rental platform fulfills its primary goal of connecting property owners and renters for long-term rentals, there are several areas identified for future improvement and expansion:

- **Online Payment Integration:** Future development will include integrating secure online payment gateways to allow renters to pay rent and service fees directly through the platform.
- **Rating and Review System:** Implementing a review and rating feature will help build trust among users by allowing renters to rate property owners and properties, and vice versa.
- **Mobile Application:** To improve accessibility and usability, especially in regions with limited desktop access, a dedicated mobile application will be developed.
- **Enhanced Filtering and Search:** Adding more advanced filters (e.g., by amenities, distance from key locations, etc.) and an interactive map-based search will help users find suitable properties more efficiently.
- **Multi-language Support:** To cater to a broader audience, especially in Ethiopia's diverse linguistic environment, the platform will be expanded to support multiple languages.
- **Admin Dashboard Enhancements:** Additional features like analytics, fraud detection tools, and customer support ticketing will be added to the admin panel for better platform management.

These improvements aim to create a more robust, user-friendly, and scalable rental platform that better serves the needs of both property owners and renters.

12. References

- [1] Airbnb, "Vacation Rentals, Cabins, Beach Houses, & More," [Online]. Available: <https://www.airbnb.com>. [Accessed: 1-Jun-2025].
- [2] Zillow, "Real Estate, Apartments, Mortgages & Home Values," [Online]. Available: <https://www.zillow.com>. [Accessed: 1-Jun-2025].
- [3] PropertyPro, "Houses for Sale, Rent and Lease in Nigeria," [Online]. Available: <https://www.propertypro.ng>. [Accessed: 1-Jun-2025].
- [4] BuyRentKenya, "Kenya Real Estate | Houses & Property for Sale and Rent," [Online]. Available: <https://www.buyrentkenya.com>. [Accessed: 1-Jun-2025].